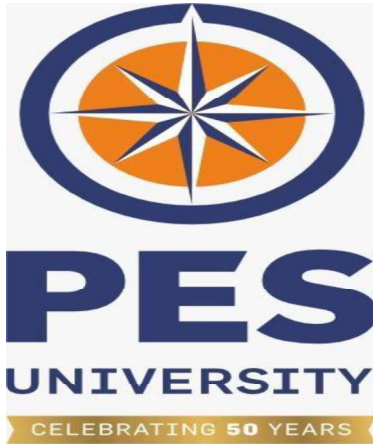




DESIGN AND ANALYSIS OF ALGORITHMS

Unit 3: B Trees

Dr. Divyashree N
Department of Computer Science & Engineering

DESIGN AND ANALYSIS OF ALGORITHMS

B Trees



DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

- Self-balancing search tree
 - Each node can contain more than one key
 - Can have more than two children.
 - All data records (or record keys) are stored at the leaves, in increasing order of the keys each parental node contains $m - 1$ ordered keys.
- It is also known as a height-balanced m-way tree.

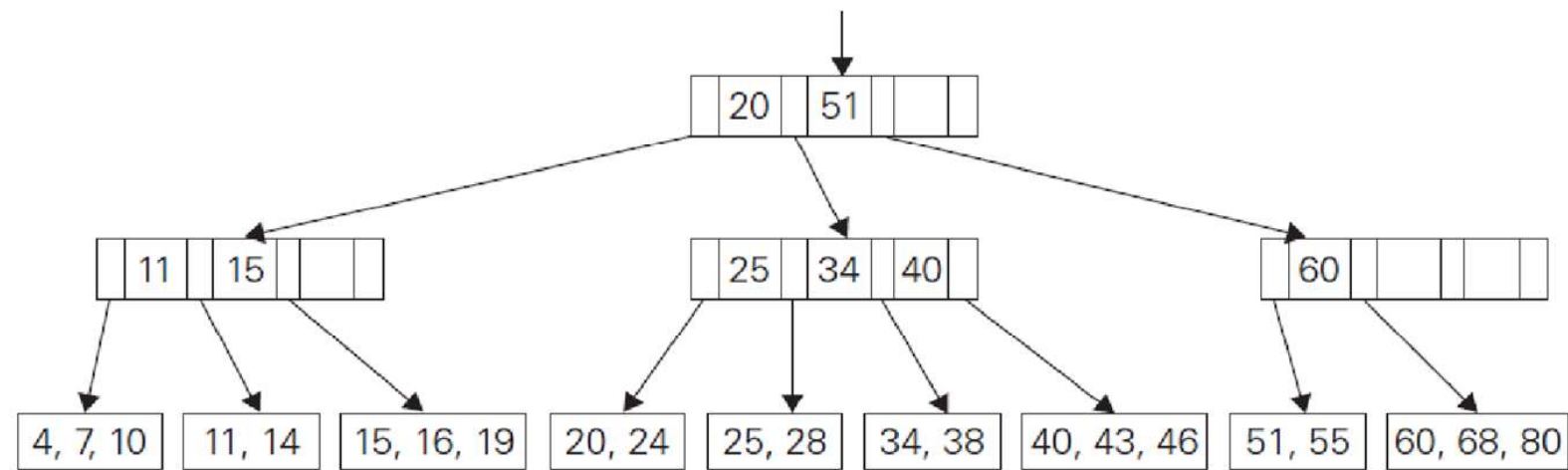


FIGURE 7.8 Example of a B-tree of order 4.

Structural properties:

- All leaves are at the same level.
- For each node x , the keys are stored in increasing order.
- If n is the order of the tree, each internal node can contain at most $m - 1$ keys along with a pointer to each child.
- Each node except root can have at most n children and at least $m/2$ children.
- All leaves have the same depth (i.e. height- h of the tree).
- The root has at least 2 children and contains a minimum of 1 key.
- If $m \geq 1$, then for any n -key B-tree of height h and minimum degree $t \geq 2$, $h \geq \log_t (m+1)/2$
- B-Tree grows and shrinks from the root which is unlike Binary Search Tree. Binary Search Trees grow downward and also shrink from downward.
- Like other balanced Binary Search Trees, time complexity to search, insert and delete is $O(\log n)$.

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

Operations on a B Tree:

- Searching
- Insertion
- Deletion

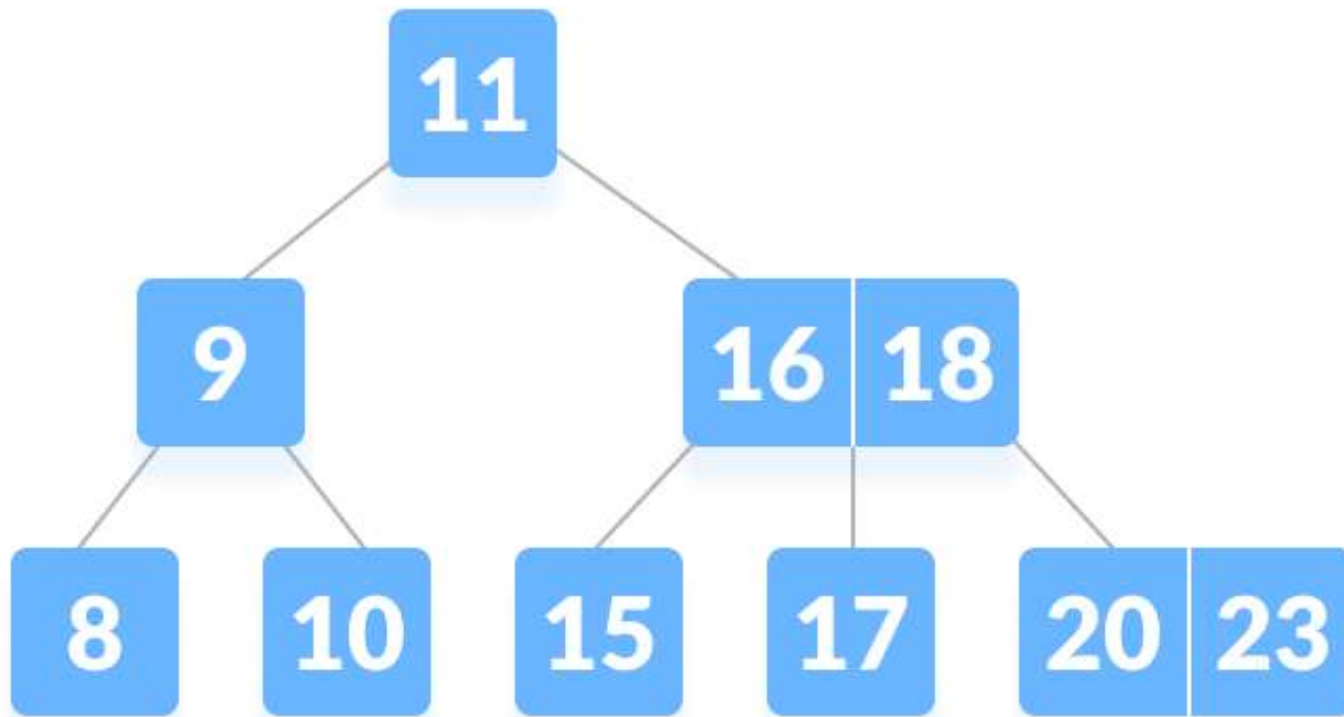
Search

1. Starting from the root node, compare k with the first key of the node.
2. If $k =$ the first key of the node, return the node and the index
3. If $k.\text{leaf} = \text{true}$, return NULL, i.e. not found
4. If $k <$ the first key of the root node, search the left child of this key recursively.
5. If there is more than one key in the current node and $k >$ the first key compare k with the next key in the node.
6. If $k <$ next key search the left child of this key (ie. k lies in between the first and the second keys). Else, search the right child of the key
7. Repeat steps 1 to 4 until the leaf is reached.

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

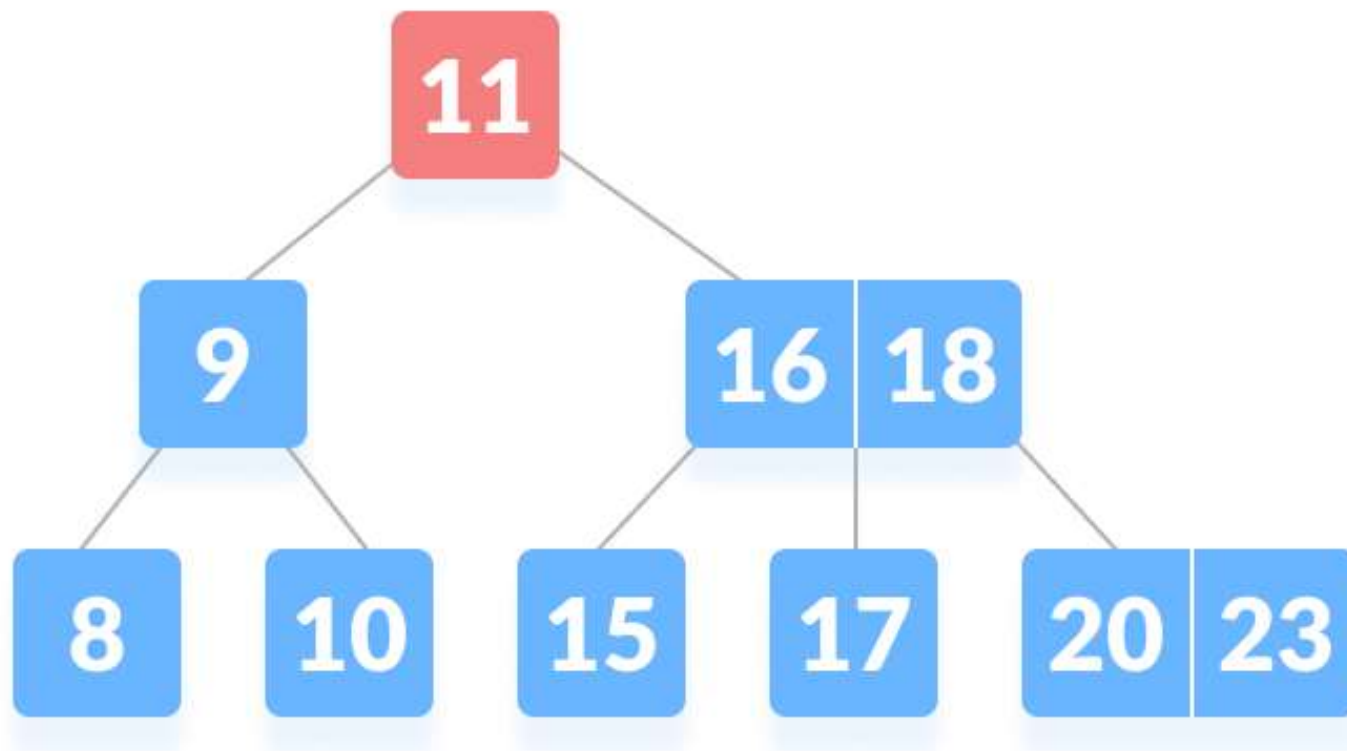
Let us search key , $k = 17$



DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

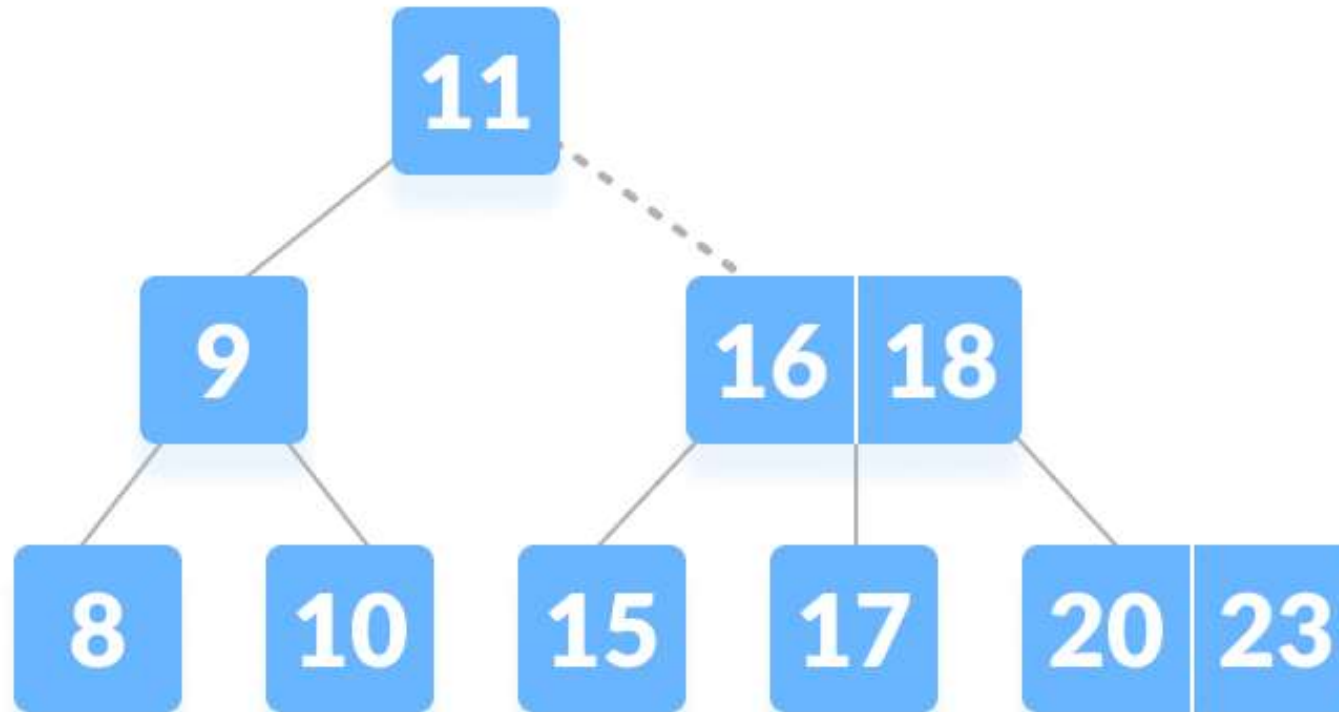
17 is not found in the root so, compare it with the root key



DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

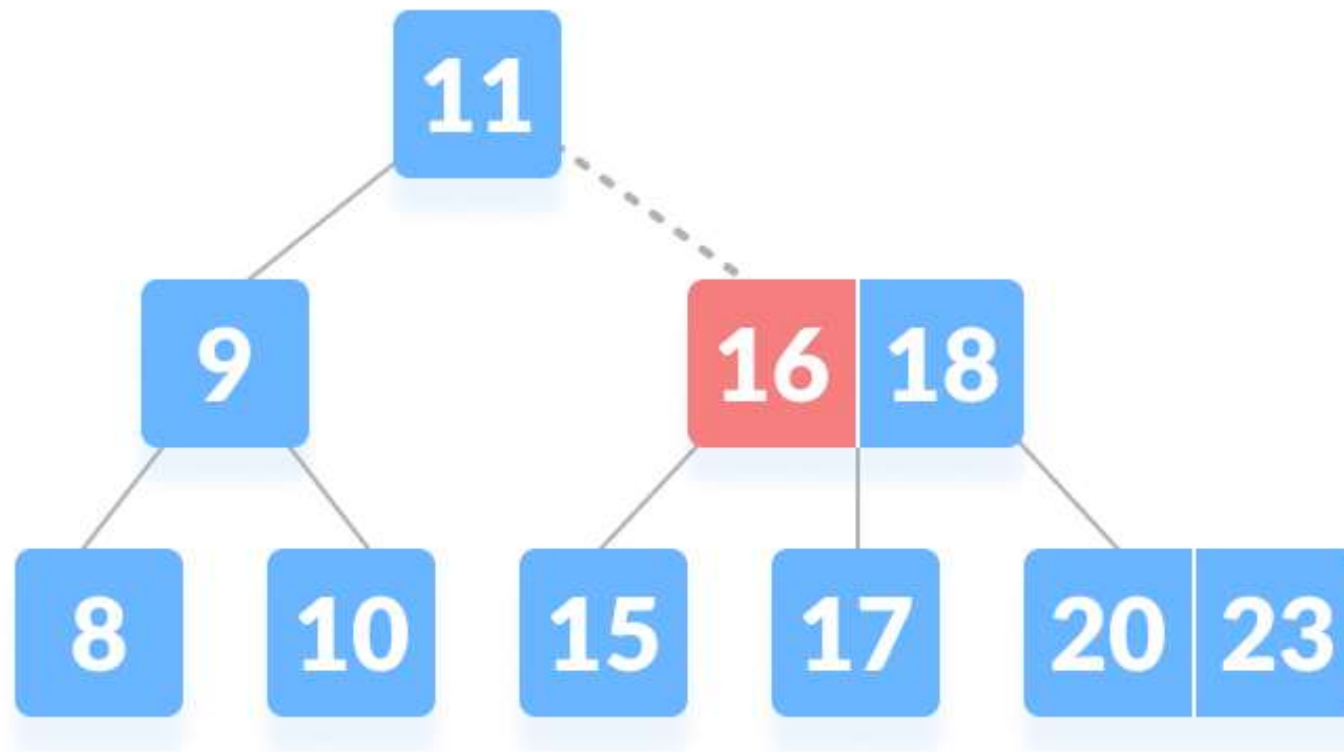
Since $k > 11$ go to the right child of the root node.



DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

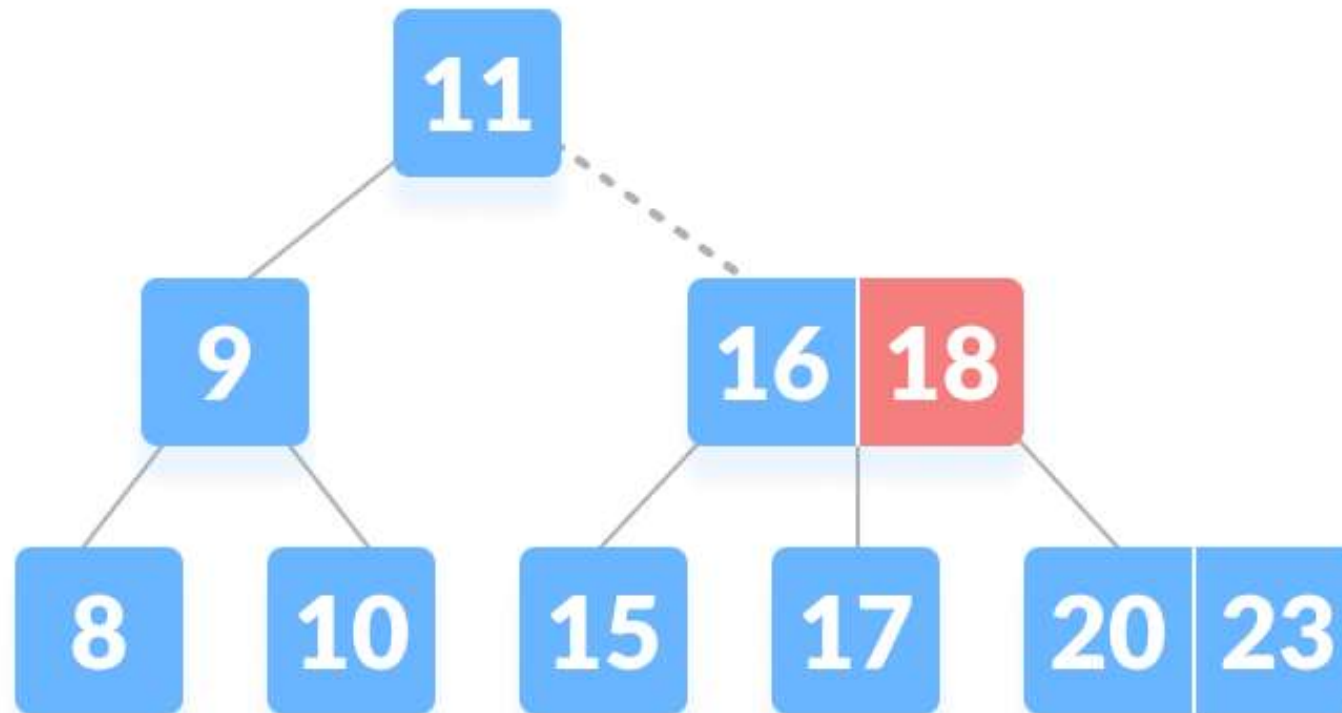
Compare k with 16. Since $k > 16$ compare k with the next key 18



DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

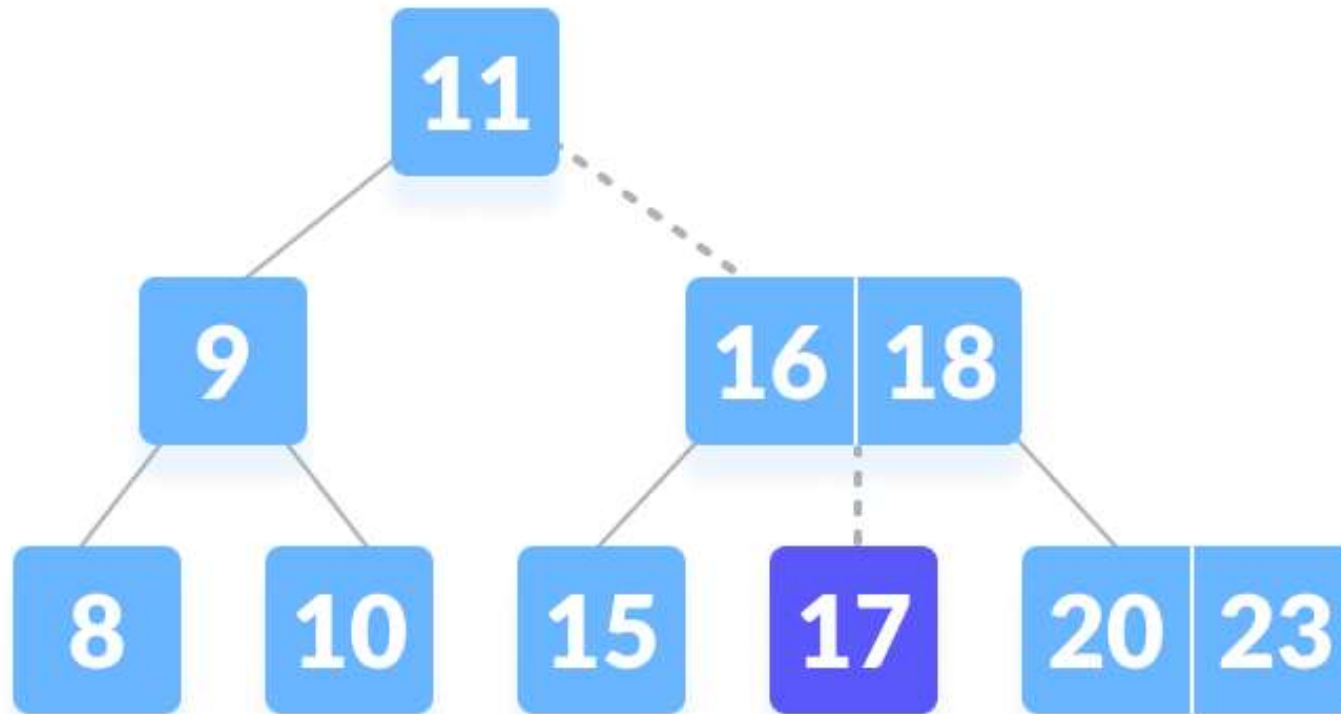
Since $k < 18$, k lies between 16 and 18. Search in the right child of 16 or the left child of 18.



DESIGN AND ANALYSIS OF ALGORITHMS

B Tree

k is found



Steps for Insertion -

- Locate the Leaf Node: Find the appropriate leaf node where the new key should be inserted.
- Insert in Sorted Order: Place the key in the correct position within the node.
- Check for Overflow: If the node exceeds the maximum allowed keys, it needs to be split.

Handling Overflow (Node Splitting)-

- Middle Key Promotion: If a node has more than $2t - 1$ keys (where t is the minimum degree), split it into two.
- Split Process:
 - The median key moves up to the parent.
 - The left and right halves form two separate nodes.
- Recursion to Root: If the parent overflows, repeat the split upwards. If the root splits, the tree height increases.

- Edge Cases
 - Root Node Split: The only case where the height of the B-Tree increases.
 - Insertion in a Full Leaf Node: Requires splitting before inserting the key.
 - Multiple Splits: Can propagate up the tree if several nodes overflow.
- Complexity Analysis
 - Search for insertion: $O(\log n)$
 - Insertion (with possible splits): $O(\log n)$
 - Space Complexity: Efficient due to high branching factor.

Minimum number of keys $\rightarrow t-1, (m/2) - 1$

Maximum number of keys $\rightarrow 2t-1, m - 1$

Maximum number of children $\rightarrow 2t, m$

Where t is the minimum degree and m is the order of the tree

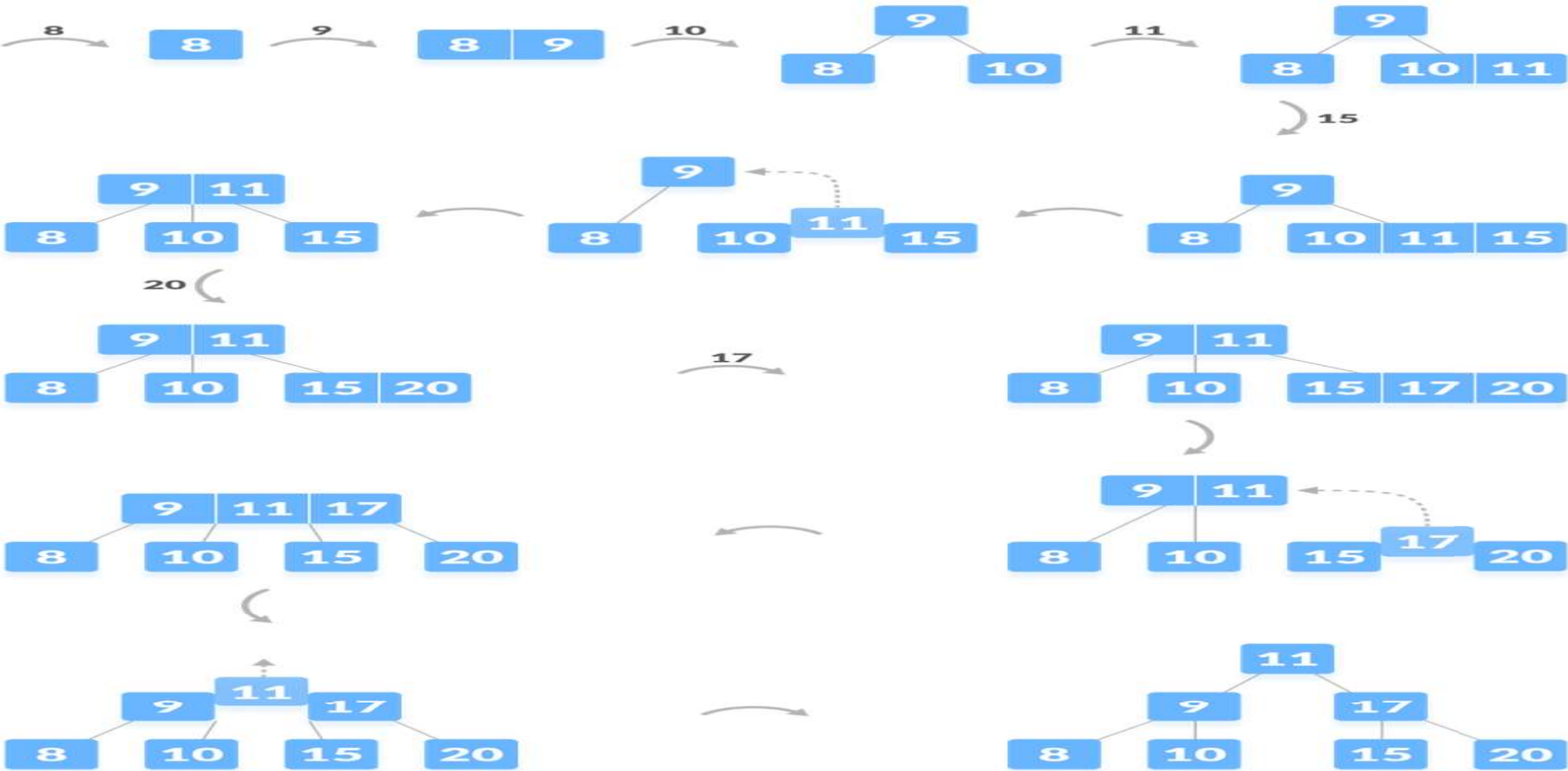
DESIGN AND ANALYSIS OF ALGORITHMS

B Tree - Insertion Algorithm

```
1: procedure B-Tree-Insert (Node x, Key k)
2:   find i such that x:keys[i] > k or i >= numkeys(x)
3:   if x is a leaf then
4:     Insert k into x.keys at i
5:   else
6:     if x:child[i] is full then
7:       Split x:child[i]
8:       if k > x:key[i] then
9:         i = i + 1
10:      end if
11:    end if
12:    B-Tree-Insert(x:child[i]; k)
13:  end if
14: end procedure
```

DESIGN AND ANALYSIS OF ALGORITHMS

B Tree Insertion Example



Insertion Example

Let us understand the insertion operation with the illustrations below.
The elements to be inserted are 8, 9, 10, 11, 15, 16, 17, 18, 20, 23.

Deletion Operation on a B-Tree:

Deleting an element on a B-tree consists of three main events: **searching the node where the key to be deleted exists**, deleting the key and balancing the tree if required.

While deleting a node in a tree, a condition called **underflow** may occur. Underflow occurs when a node contains less than the minimum number of keys it should hold.

The terms to be understood before studying deletion operation are:

1. Inorder Predecessor

The largest key on the left child of a node is called its inorder predecessor.

2. Inorder Successor

The smallest key on the right child of a node is called its inorder successor.

Before going through the steps below, one must know these facts about a B tree of degree **m**.

1. A node can have a maximum of m children. (i.e. 3)
2. A node can contain a maximum of $m - 1$ keys. (i.e. 2)
3. A node should have a minimum of $\lceil m/2 \rceil$ children. (i.e. 2)
4. A node (except root node) should contain a minimum of $\lceil m/2 \rceil - 1$ keys. (i.e. 1)

There are three main cases for deletion operation in a B tree.

DESIGN AND ANALYSIS OF ALGORITHMS

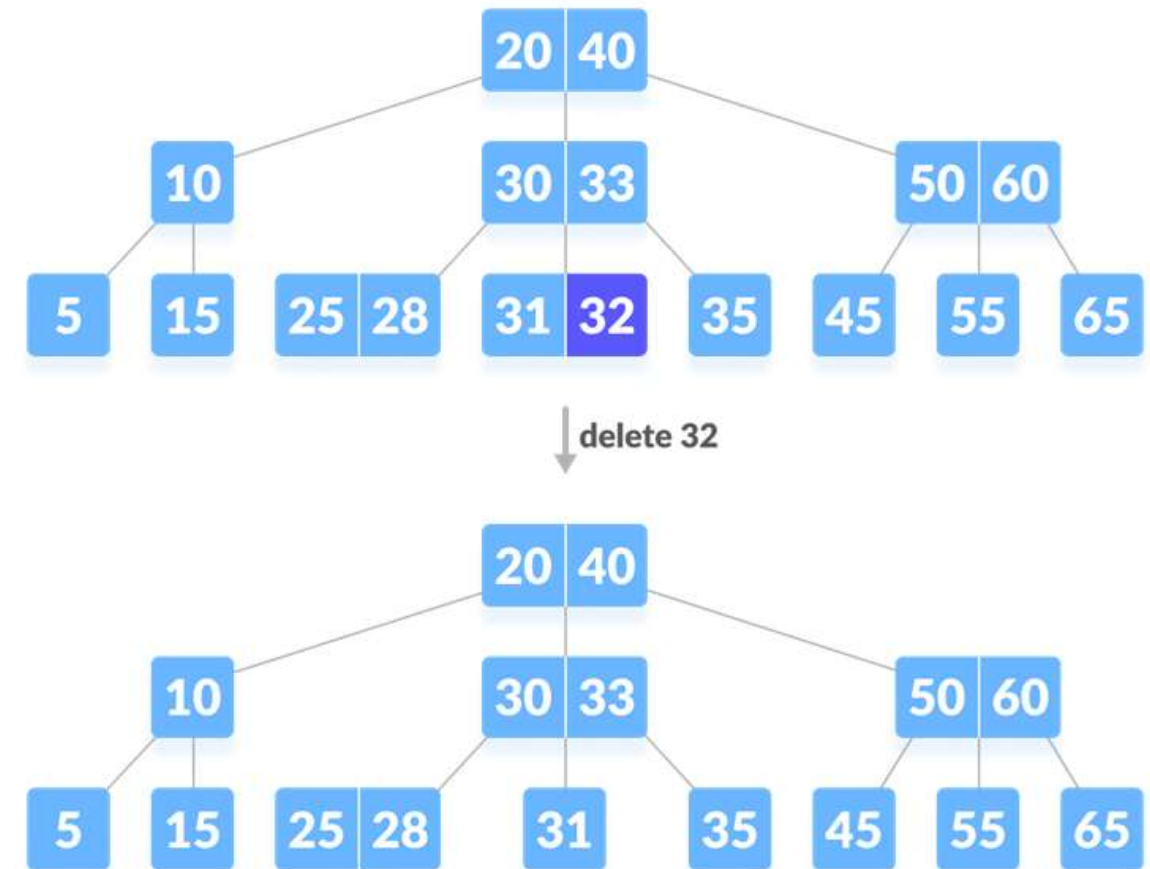
B Tree - Deletion

Case I

The key to be deleted lies in the leaf. There are two cases for it.

1. The deletion of the key does not violate the property of the minimum number of keys a node should hold.

In the tree below, deleting 32 does not violate the above properties.



DESIGN AND ANALYSIS OF ALGORITHMS

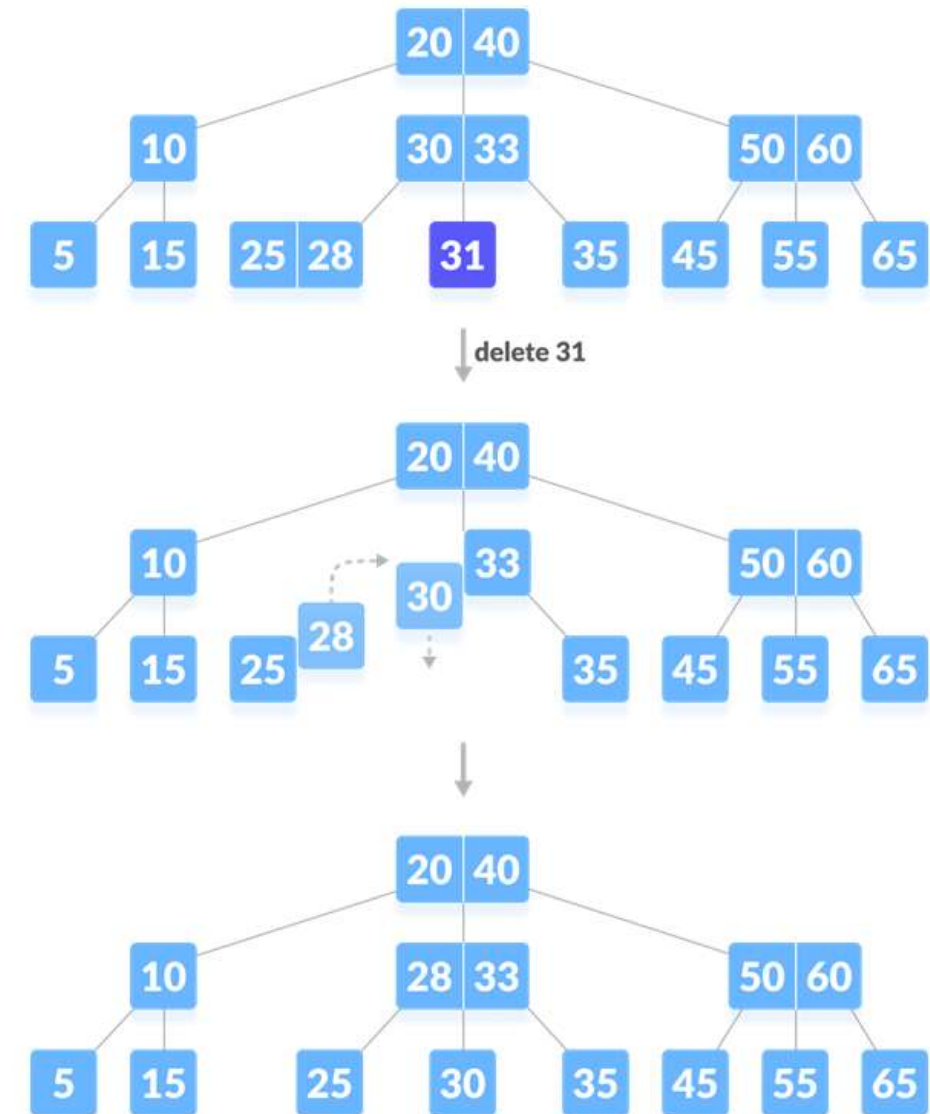
B Tree - Deletion

2. The deletion of the key violates the property of the minimum number of keys a node should hold. In this case, we borrow a key from its immediate neighboring sibling node in the order of left to right.

First, visit the immediate left sibling. If the left sibling node has more than a minimum number of keys, then borrow a key from this node.

Else, check to borrow from the immediate right sibling node.

In the tree below, deleting 31 results in the above condition. Let us borrow a key from the left sibling node.

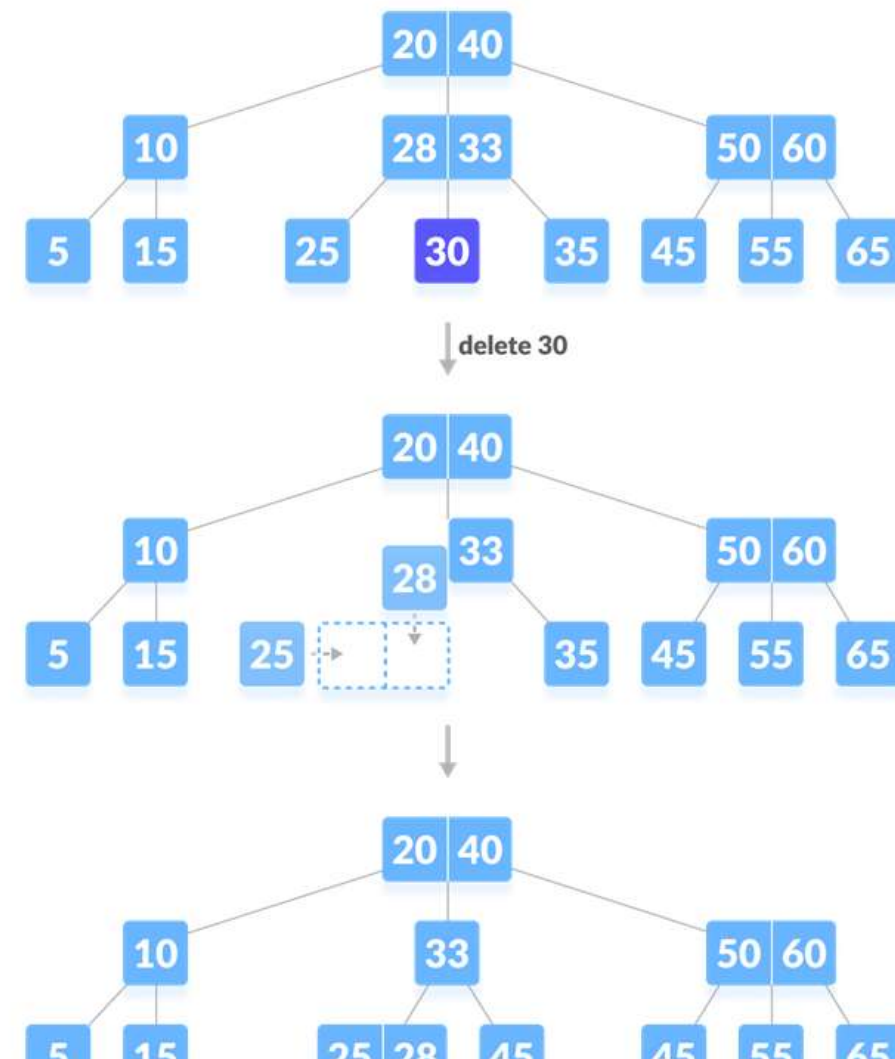


DESIGN AND ANALYSIS OF ALGORITHMS

B Tree - Deletion

If both the immediate sibling nodes already have a minimum number of keys, then merge the node with either the left sibling node or the right sibling node. **This merging is done through the parent node.**

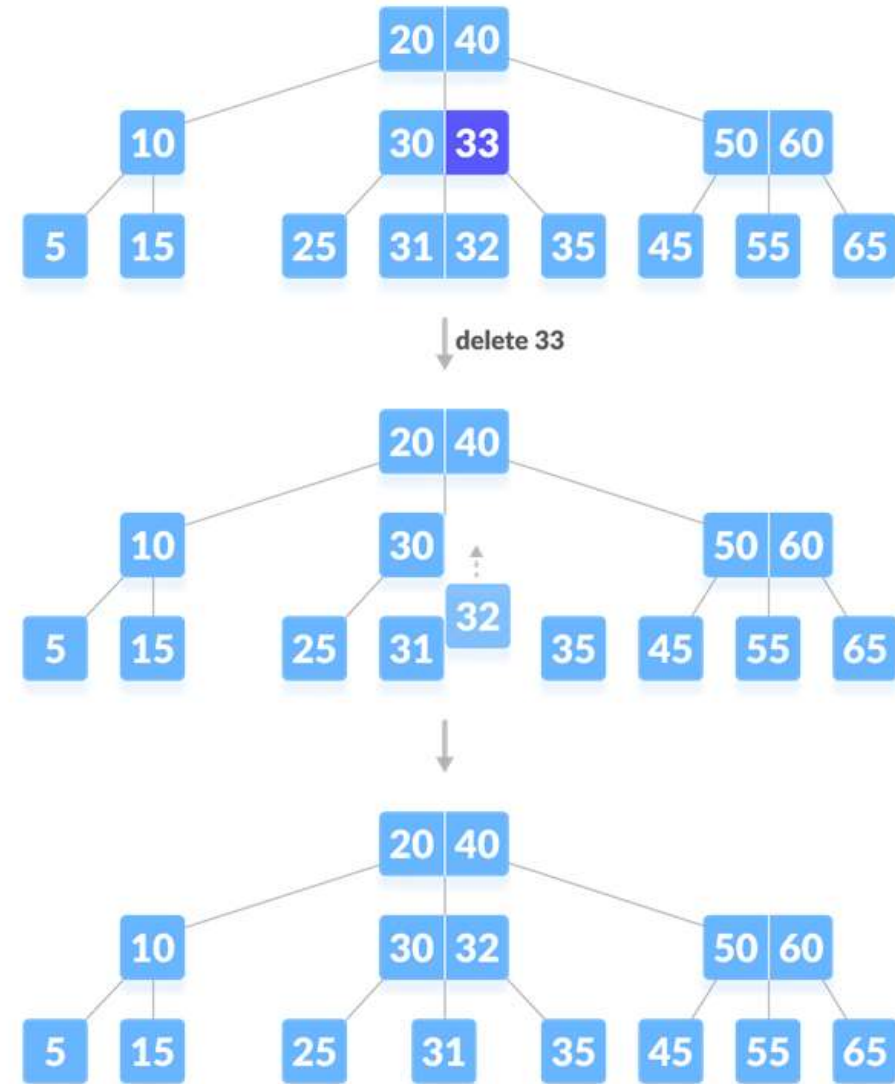
Deleting 30 results in the above case.



Case II

If the key to be deleted lies in the internal node, the following cases occur.

1. The internal node, which is deleted, is replaced by an inorder predecessor if the left child has more than the minimum number of keys.



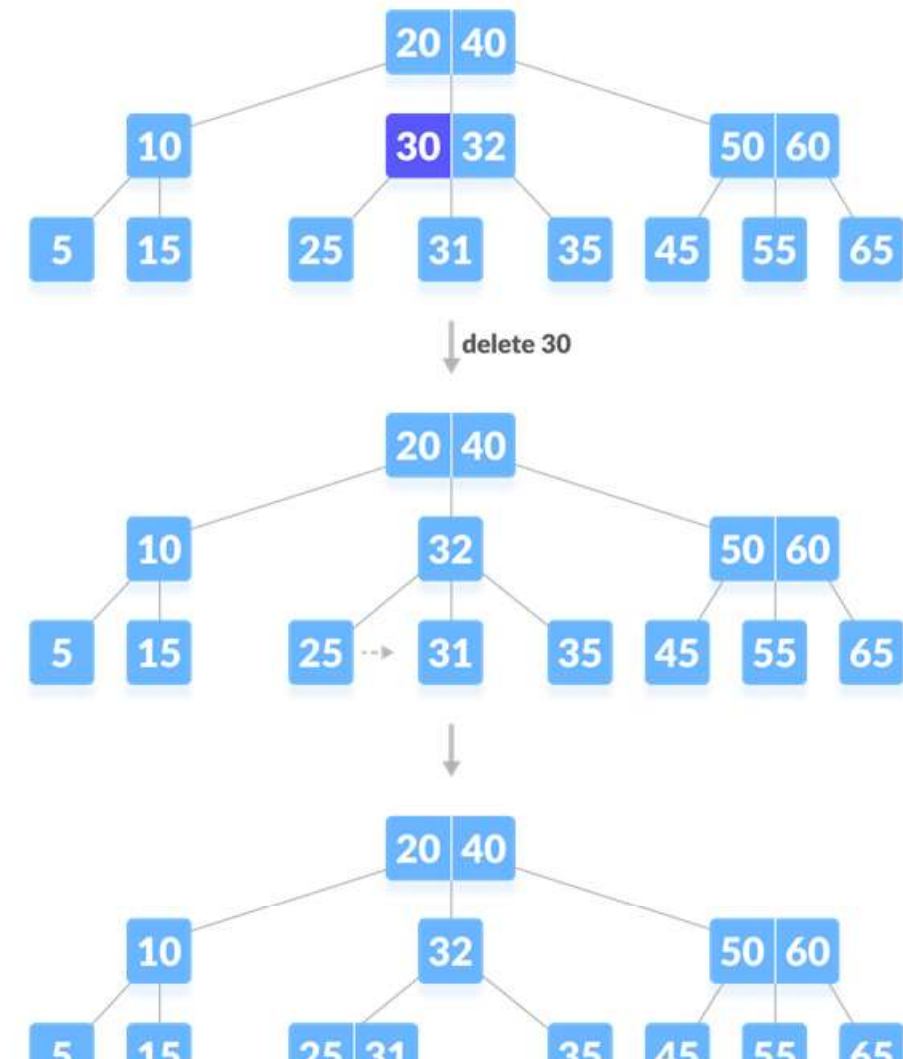
DESIGN AND ANALYSIS OF ALGORITHMS

B Tree - Deletion

2. The internal node, which is deleted, is replaced by an inorder successor if the right child has more than the minimum number of keys.

3. If either child has exactly a minimum number of keys then, merge the left and the right children.

After merging if the parent node has less than the minimum number of keys then, look for the siblings as in Case I.



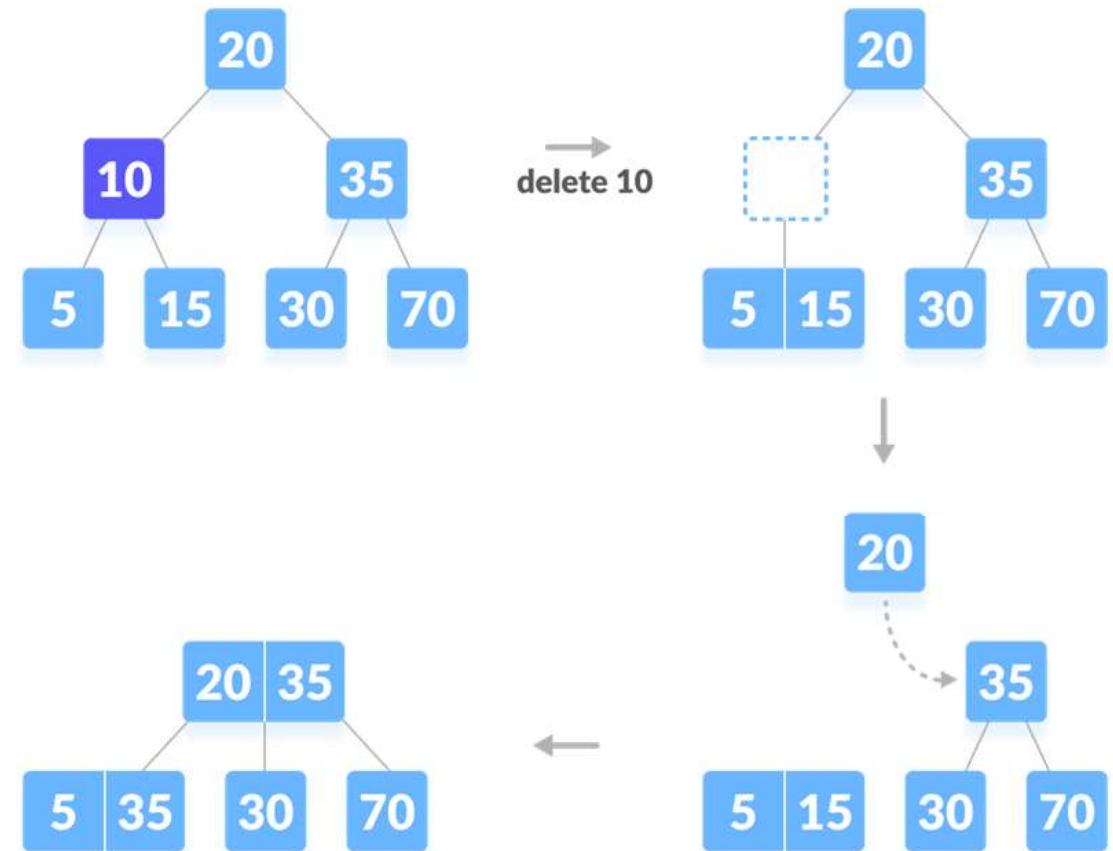
DESIGN AND ANALYSIS OF ALGORITHMS

B Tree - Deletion

Case III

In this case, the height of the tree shrinks. If the target key lies in an internal node, and the deletion of the key leads to a fewer number of keys in the node (i.e. less than the minimum required), then look for the inorder predecessor and the inorder successor. If both the children contain a minimum number of keys then, borrowing cannot take place. This leads to Case II(3) i.e. merging the children.

Again, look for the sibling to borrow a key. But, if the sibling also has only a minimum number of keys then, merge the node with the sibling along with the parent. Arrange the children accordingly (increasing order).



Deletion Complexity

Best case Time complexity: $\Theta(\log n)$

Average case Space complexity: $\Theta(n)$

Worst case Space complexity: $\Theta(n)$